DevOps Explanation Request

Alright vicky 😈

Next logical step unlocked:

Kubernetes ServiceAccount – RBAC (THE POD IDENTITY STORY)

Same flow as before. Same mental clarity. Same production realism.

☠️ Way (Before ServiceAccounts / or using wrong identity)

- Pods used **admin kubeconfig** ×
- Hard-coded tokens inside containers ×
- One compromised pod = cluster compromise ☠️

Classic rookie mistake:

> "It's just a pod, what can it do?"
> Answer: *Delete your cluster if you let it.*

💣 Problem

1. Pods also talk to **Kubernetes API**
2. Pods need:
   - list pods
   - read configmaps
   - watch resources
3. But pods are **NOT humans**
4. Using human certs / admin access is dangerous

So Kubernetes asks:

> "If a pod calls me… **WHO is it?**
> And **WHAT is it allowed to do?**"

🛠️ How Kubernetes Solves This

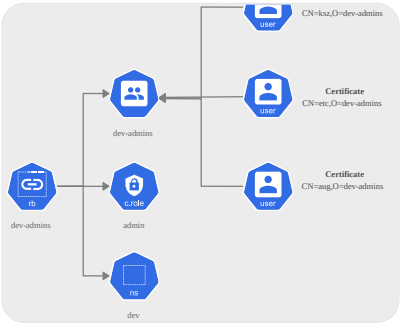
Using **ServiceAccount + Token + RBAC**

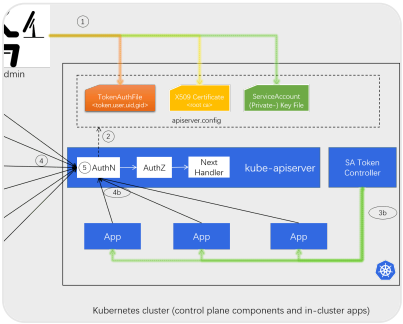
```rust
ServiceAccount → Identity for pods
RBAC           → Permissions for that identity
```

Think of it as:

> **ServiceAccount = user for pods**

Kubernetes does **NOT** use certificates for pods.
Pods use **tokens**.





Step 1 – High-level Overview

Human vs Pod identity

Who	Authentication
Human user	Certificates
Pod	ServiceAccount token

RBAC doesn't care **who** you are
RBAC only cares **what identity name** you have.

Step 2 – What is a ServiceAccount?

A **ServiceAccount** is:

- A Kubernetes object
- Namespace-scoped

- Used by Pods to authenticate to API server

Every namespace already has one:

```bash
kubectl get sa
```

Output:

```cpp
default
```

⚠️ If you don't specify a ServiceAccount, Pod uses **default** (bad idea).

Step 3 – Real Scenario (Why we need it)

Scenario

- Pod needs to **list pods** in `default` namespace
- Nothing else
- No admin powers

So we will:

1. Create a ServiceAccount
2. Create a Role
3. Bind Role to ServiceAccount
4. Attach ServiceAccount to Pod

Step 4 – Create ServiceAccount (IDENTITY)

📄 `sa.yaml`

```yaml
apiVersion: v1
kind: ServiceAccount
metadata:
```

```yaml
  name: pod-reader-sa
  namespace: default
```

Apply it:

```bash
kubectl apply -f sa.yaml
```

Verify:

```bash
kubectl get sa
kubectl describe sa pod-reader-sa
```

At this point:

× No permissions

× Identity only

Step 5 – Create Role (WHAT is allowed)

📄 `role.yaml`

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader-role
  namespace: default
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
```

Apply:

```bash
kubectl apply -f role.yaml
```

Step 6 – RoleBinding (WHO gets permission)

Now we bind **ServiceAccount**, not user.

📄 `rolebinding.yaml`

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: pod-reader-binding
  namespace: default
subjects:
- kind: ServiceAccount
  name: pod-reader-sa
  namespace: default
roleRef:
  kind: Role
  name: pod-reader-role
  apiGroup: rbac.authorization.k8s.io
```

Apply:

```bash
kubectl apply -f rolebinding.yaml
```

Step 7 – Verify RBAC (Before attaching to Pod)

Use impersonation:

```bash
kubectl auth can-i get pods \
--as system:serviceaccount:default:pod-reader-sa
```

Result:

```bash
```

```
yes
```

Try forbidden action:

```bash
kubectl auth can-i delete pods \
--as system:serviceaccount:default:pod-reader-sa
```

Result:

```perl
no
```

Perfect.

Step 8 – Attach ServiceAccount to Pod

📄 `pod.yaml`

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: test-pod
spec:
  serviceAccountName: pod-reader-sa
  containers:
  - name: test
    image: busybox
    command: ["sh", "-c", "sleep 3600"]
```

Apply:

```bash
kubectl apply -f pod.yaml
```

Step 9 – How Pod Authenticates Internally (IMPORTANT)

Inside the pod:

```bash
/var/run/secrets/kubernetes.io/serviceaccount/
```

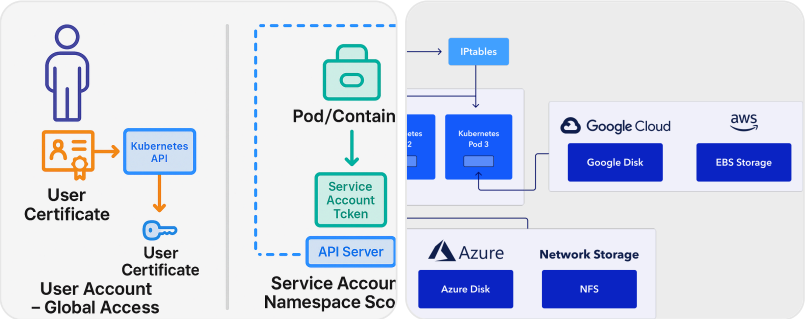
Contains:

File	Meaning
`token`	🔑 JWT token (identity)
`ca.crt`	CA certificate
`namespace`	Pod namespace

The pod uses:

- token → authentication
- RBAC → authorization

No certs. No kubeconfig.



Step 10 – Test from Inside Pod

```bash
kubectl exec -it test-pod -- sh
```

Try calling API (example):

```bash
```

```
kubectl get pods
```

✅ Allowed

Try delete:

```bash
kubectl delete pod nginx
```

× Forbidden

RBAC enforced **inside pod**

Step 11 – ServiceAccount vs User (Never Confuse Again)

Feature	User	ServiceAccount
Used by	Humans	Pods
Auth method	Certificate	Token
Scope	Cluster-wide	Namespace
kubeconfig	Yes	No
Auto-mounted	×	✅

Step 12 – Common Mistakes (Seen DAILY)

× Using default ServiceAccount
× Giving cluster-admin to ServiceAccount
× Forgetting RoleBinding
× Binding Role to User instead of ServiceAccount
× Assuming Pod has no cluster access

🧠 Final Mental Model (Tattoo-level)

```sql
User            → cert → RBAC
ServiceAccount  → token → RBAC
```

Or simply:

> **Pods authenticate as ServiceAccounts.**
> **ServiceAccounts are controlled by RBAC.**

If you want next 🔥

- ClusterRole + ServiceAccount
- Why default SA is dangerous
- IRSA (AWS) / Workload Identity (GCP)
- Debugging “forbidden” from inside pod
- CKA-style ServiceAccount traps

Say the word 😏